

Билет 3. Проблема доступа к общим данным.

`std::mutex`, `std::shared_mutex`,
`std::recursive_mutex`, `std::unique_lock`,
`std::shared_lock`, `std::lock`

0. Формулировка билета

Проблема доступа к общим данным. Примитивы синхронизации: `std::mutex`,
`std::shared_mutex`, `std::recursive_mutex`, `std::unique_lock`, `std::shared_lock`.
Функция `std::lock`.

Главная идея билета:

Когда несколько потоков обращаются к общим изменяемым данным, нужно синхронизировать доступ. Иначе возникают data race, повреждение данных, неопределённое поведение, deadlock-и и другие проблемы.

Минимальная схема:

```
несколько потоков
    ↓
общие изменяемые данные
    ↓
риск одновременного доступа
    ↓
mutex / shared_mutex / lock objects
```

1. Почему эта тема возникает после многопоточного сервера

В билете 2 мы сделали сервер, где на каждого клиента создаётся отдельный поток.

```
main thread:
  accept client #1 → thread #1
  accept client #2 → thread #2
  accept client #3 → thread #3
```

Пока каждый поток работает только со своим `client_fd`, всё относительно просто.

Но в реальном сервере часто появляются общие данные:

- общий счётчик запросов
- общий лог
- общий кэш
- общая очередь задач
- общая база активных клиентов
- общая статистика

Например:

```
std::unordered_map<long long, bool> cache;
```

Если несколько потоков одновременно читают и изменяют этот `cache`, программа становится некорректной.

2. Пример: сервер проверки простоты чисел с кэшем

Пусть клиенты отправляют числа, а сервер проверяет, является ли число простым.

Чтобы не считать одно и то же несколько раз, заведём кэш:

```
std::unordered_map<long long, bool> cache;
```

Наивная обработка:

```
std::unordered_map<long long, bool> cache;

bool is_prime(long long n) {
    if (n <= 1) {
        return false;
    }

    if (n == 2) {
        return true;
    }

    if (n % 2 == 0) {
        return false;
    }

    for (long long d = 3; d * d <= n; d += 2) {
        if (n % d == 0) {
            return false;
        }
    }
}
```

```

        return true;
    }

    bool get_result(long long number) {
        auto it = cache.find(number);

        if (it != cache.end()) {
            return it->second;
        }

        bool result = is_prime(number);
        cache[number] = result;

        return result;
    }

```

В однопоточном сервере это может работать.

В многопоточном — нет.

3. Что может пойти не так без синхронизации

Представим два потока:

```

thread #1: get_result(1000003)
thread #2: get_result(1000003)

```

Оба обращаются к одному `std::unordered_map`.

Проблема не только в том, что они могут дважды посчитать одно и то же.

Гораздо хуже:

```

один поток читает hash table
другой поток в это время вставляет элемент
при вставке unordered_map может сделать rehash
первый поток продолжает работать со старой внутренней структурой

```

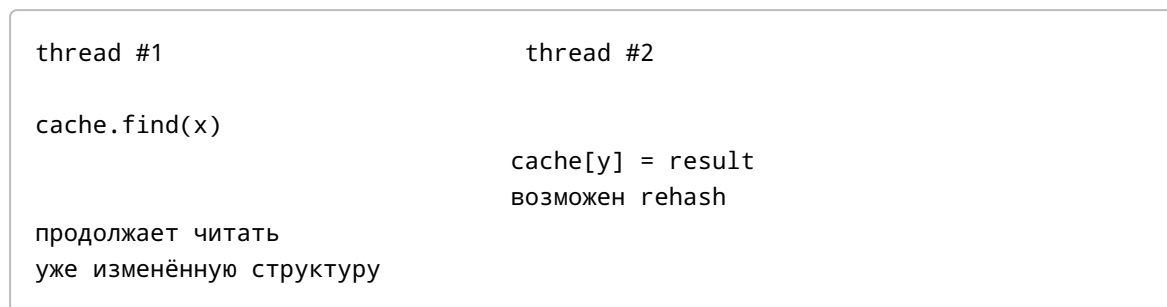
Результат:

```

неправильные данные
падение программы
повреждение памяти
undefined behavior

```

Схема:

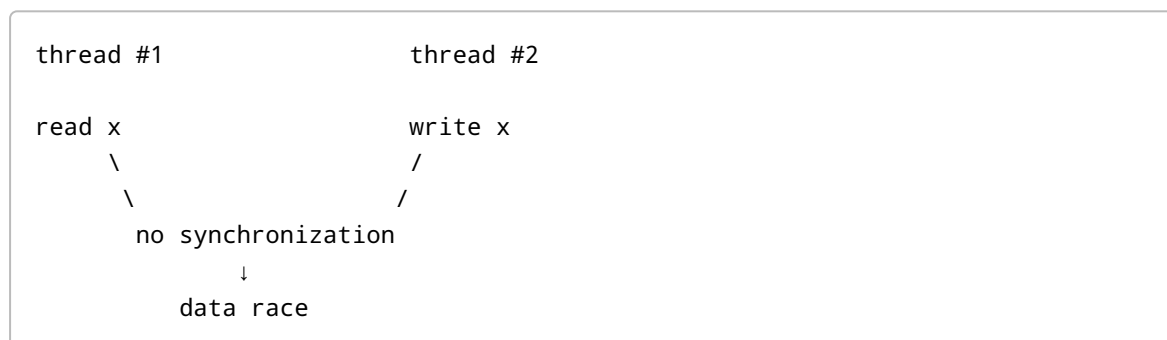


4. Data race

Data race — это ситуация, когда:

1. два или более потока обращаются к одной и той же области памяти;
2. хотя бы одно обращение — запись;
3. между этими обращениями нет синхронизации.

Схема:



В C++ data race приводит к **undefined behavior**.

Это значит, что программа не просто “может дать неправильный ответ”. С точки зрения стандарта она вообще не имеет определённой семантики.

5. Race condition vs data race

Полезно различать два термина.

Data race

Формальное понятие из модели памяти C++.

Пример:

```
int counter = 0;

void increment() {
    ++counter;
}
```

Если два потока одновременно вызывают `increment`, возникает data race.

Почему?

`++counter` не является одной неделимой операцией.

Обычно это:

```
read counter
add 1
write counter
```

Два потока могут потерять обновление.

Race condition

Более широкое понятие: результат программы зависит от порядка выполнения потоков.

Race condition может быть даже без data race.

Например, если все операции защищены мьютексом, но логика зависит от того, кто первым взял мьютекс.

Пример:

```
std::mutex m;
int winner = -1;

void try_win(int id) {
    std::lock_guard<std::mutex> lock(m);

    if (winner == -1) {
        winner = id;
    }
}
```

Data race нет, но есть race condition: кто первым войдёт в критическую секцию, тот станет победителем.

6. Критическая секция

Критическая секция — участок кода, где поток работает с общими данными, которые нельзя одновременно менять из нескольких потоков.

Пример:

```
{
    std::lock_guard<std::mutex> lock(cache_mutex);

    auto it = cache.find(number);

    if (it == cache.end()) {
        cache[number] = is_prime(number);
    }
}
```

Схема:

```
lock mutex
↓
работа с shared data
↓
unlock mutex
```

Цель мьютекса — сделать так, чтобы одновременно в критической секции был только один поток.

7. `std::mutex`

`std::mutex` — базовый примитив взаимного исключения.

Он имеет методы:

```
lock()
unlock()
try_lock()
```

`lock`

Захватывает мьютекс.

Если мьютекс свободен — поток входит дальше.

Если мьютекс уже занят другим потоком — текущий поток блокируется.

```
mutex.lock();
```

`unlock`

Освобождает мьютекс.

```
mutex.unlock();
```

`try_lock`

Пытается захватить мьютекс без блокировки.

```
if (mutex.try_lock()) {  
    // получилось захватить  
    mutex.unlock();  
} else {  
    // мьютекс занят, но мы не заблокировались  
}
```

Обычно руками `lock` / `unlock` писать не стоит, потому что легко забыть `unlock`, особенно при исключениях.

8. Наивный код с ручным `lock` / `unlock`

```
std::mutex cache_mutex;  
std::unordered_map<long long, bool> cache;  
  
bool get_result(long long number) {  
    cache_mutex.lock();  
  
    auto it = cache.find(number);  
  
    if (it != cache.end()) {  
        bool result = it->second;  
        cache_mutex.unlock();  
        return result;  
    }  
  
    bool result = is_prime(number);  
    cache[number] = result;  
  
    cache_mutex.unlock();  
}
```

```
    return result;
}
```

Проблемы:

1. Можно забыть `unlock`.
2. Можно выйти из функции раньше и не освободить мьютекс.
3. Исключение может прервать функцию до `unlock`.
4. Код становится хрупким.

Например:

```
cache_mutex.lock();

throw std::runtime_error("oops");

cache_mutex.unlock(); // не выполнится
```

Мьютекс останется захваченным.

9. RAII и `std::lock_guard`

В C++ обычно используют RAII.

RAII — Resource Acquisition Is Initialization.

Идея:

```
ресурс захватывается в конструкторе объекта
ресурс освобождается в деструкторе объекта
```

Для мьютекса простейший RAII-объект — `std::lock_guard`.

```
std::mutex cache_mutex;

void f() {
    std::lock_guard<std::mutex> lock(cache_mutex);

    // критическая секция
}
```

Когда объект `lock` создаётся, он вызывает:


```
cache_mutex.lock();
```

Когда `lock` выходит из области видимости, он вызывает:

```
cache_mutex.unlock();
```

Схема:

```
{  
    lock_guard создан → mutex.lock()  
  
    работа с общими данными  
  
}    lock_guard уничтожен → mutex.unlock()
```

10. Исправленный кэш через `std::mutex`

```
std::unordered_map<long long, bool> cache;  
std::mutex cache_mutex;  
  
bool get_result(long long number) {  
    std::lock_guard<std::mutex> lock(cache_mutex);  
  
    auto it = cache.find(number);  
  
    if (it != cache.end()) {  
        return it->second;  
    }  
  
    bool result = is_prime(number);  
    cache[number] = result;  
  
    return result;  
}
```

Теперь одновременно только один поток может работать с `cache`.

Плюс:

```
простота  
корректность  
нет data race
```

Минус:

все обращения сериализуются
даже чтения не могут идти параллельно

11. Почему нельзя держать мьютекс слишком долго

В коде выше мы считаем `is_prime(number)` под мьютексом.

```
std::lock_guard<std::mutex> lock(cache_mutex);

bool result = is_prime(number);
cache[number] = result;
```

Если `is_prime` работает долго, все остальные потоки ждут.

Лучше уменьшать критическую секцию.

```
bool get_result(long long number) {
    {
        std::lock_guard<std::mutex> lock(cache_mutex);

        auto it = cache.find(number);

        if (it != cache.end()) {
            return it->second;
        }
    }

    bool result = is_prime(number);

    {
        std::lock_guard<std::mutex> lock(cache_mutex);
        cache[number] = result;
    }

    return result;
}
```

Теперь тяжёлое вычисление выполняется без мьютекса.

Но появляется другой эффект:

два потока могут одновременно не найти число в кэше
оба посчитают `is_prime`
оба запишут результат

Это не data race, если запись защищена мьютексом.

Но это лишняя работа.

12. Double-check после вычисления

Можно сделать повторную проверку перед вставкой.

```
bool get_result(long long number) {
    {
        std::lock_guard<std::mutex> lock(cache_mutex);

        auto it = cache.find(number);

        if (it != cache.end()) {
            return it->second;
        }
    }

    bool result = is_prime(number);

    {
        std::lock_guard<std::mutex> lock(cache_mutex);

        auto [it, inserted] = cache.emplace(number, result);

        return it->second;
    }
}
```

Если другой поток успел вставить результат, `emplace` не перезапишет старое значение.

Смысл:

1. быстро проверили под lock
2. долго посчитали без lock
3. аккуратно вставили под lock

13. `std::unique_lock`

`std::unique_lock` — более гибкий RAII-объект для владения мьютексом.

Он похож на `lock_guard`, но умеет больше:

```
можно явно unlock()
можно снова lock()
можно отложить захват через defer_lock
можно передавать владение move-ом
нужен для condition_variable
```

Простой пример:

```
std::unique_lock<std::mutex> lock(cache_mutex);

// критическая секция
```

Такой код похож на `lock_guard`.

Но можно сделать:

```
std::unique_lock<std::mutex> lock(cache_mutex);

// работа под мьютексом

lock.unlock();

// долгая операция без мьютекса

lock.lock();

// снова работа под мьютексом
```

14. `lock_guard` vs `unique_lock`

Свойство	<code>std::lock_guard</code>	<code>std::unique_lock</code>
Захватывает в конструкторе	да	обычно да
Освобождает в деструкторе	да	да
Можно явно <code>unlock</code>	нет	да
Можно явно <code>lock</code> снова	нет	да

Свойство	<code>std::lock_guard</code>	<code>std::unique_lock</code>
Можно создать без захвата	нет	да, <code>std::defer_lock</code>
Можно перемещать	нет	да
Нужен для <code>condition_variable</code>	нет	да
Накладные расходы	минимальные	чуть больше

Правило:

если нужен просто lock/unlock по области видимости → `lock_guard`
 если нужна гибкость → `unique_lock`

15. Пример раннего освобождения через `unique_lock`

Допустим, мы хотим взять из очереди задачу, а потом выполнить её без удержания мьютекса.

```
std::queue<std::function<void()>> tasks;
std::mutex tasks_mutex;

void worker() {
    std::unique_lock<std::mutex> lock(tasks_mutex);

    auto task = std::move(tasks.front());
    tasks.pop();

    lock.unlock();

    task();
}
```

Почему `task()` надо выполнять без мьютекса?

Потому что задача может быть долгой, может делать I/O, может пытаться взять другие мьютексы.

Если выполнять `task()` под `tasks_mutex`, остальные потоки не смогут брать задачи из очереди.

16. `std::shared_mutex`

`std::shared_mutex` — мьютекс для сценария “много читателей, мало писателей”.

Он поддерживает два режима:

1. **shared lock** — разделяемая блокировка для чтения;
2. **unique lock** — эксклюзивная блокировка для записи.

Схема совместимости:

```
reader + reader → можно  
reader + writer → нельзя  
writer + writer → нельзя
```

Таблица:

Кто держит lock	Кто хочет войти	Можно?
reader	reader	да
reader	writer	нет
writer	reader	нет
writer	writer	нет

17. `std::shared_lock`

`std::shared_lock` — RAII-объект для разделяемого чтения.

```
std::shared_mutex cache_mutex;  
std::unordered_map<long long, bool> cache;  
  
bool contains(long long number) {  
    std::shared_lock lock(cache_mutex);  
  
    return cache.find(number) != cache.end();  
}
```

Несколько потоков могут одновременно держать `std::shared_lock` на одном `shared_mutex`.

18. Запись в `shared_mutex`: `std::unique_lock`

Для записи нужен эксклюзивный lock.

```
void put(long long number, bool result) {  
    std::unique_lock lock(cache_mutex);
```

```
    cache[number] = result;
}
```

Пока один поток держит `unique_lock`:

никто другой не может ни читать, ни писать

19. Кэш через `std::shared_mutex`

```
#include <shared_mutex>
#include <unordered_map>

std::unordered_map<long long, bool> cache;
std::shared_mutex cache_mutex;

bool get_result(long long number) {
    {
        std::shared_lock lock(cache_mutex);

        auto it = cache.find(number);

        if (it != cache.end()) {
            return it->second;
        }
    }

    bool result = is_prime(number);

    {
        std::unique_lock lock(cache_mutex);

        auto [it, inserted] = cache.emplace(number, result);

        return it->second;
    }
}
```

Что здесь происходит:

1. Читаем кэш под `shared_lock`.
Много читателей могут делать это одновременно.
2. Если не нашли — считаем без `lock`.

3. Записываем под `unique_lock`.
Запись эксклюзивная.

Плюсы:

чтения не блокируют друг друга
лучше для read-heavy workloads

Минусы:

сложнее
возможны повторные вычисления
можно столкнуться с starvation

20. Проблема starvation в `shared_mutex`

`std::shared_mutex` может страдать от голодания.

Сценарий:

читатели приходят постоянно
писатель ждёт `unique_lock`
новые читатели снова получают `shared_lock`
писатель всё ещё ждёт

Если реализация даёт приоритет читателям, писатель может ждать очень долго.

Стандарт C++ не гарантирует конкретную политику fairness для `std::shared_mutex`.

Поэтому `shared_mutex` не всегда автоматически лучше обычного `mutex`.

Правило:

`shared_mutex` полезен, если чтений действительно много,
записей мало,
а критическая секция достаточно дорогая

Если критическая секция маленькая, обычный `mutex` может быть быстрее.

21. `std::recursive_mutex`

Обычный `std::mutex` нельзя захватить дважды из одного и того же потока.


```
std::mutex m;

void f() {
    std::lock_guard<std::mutex> lock(m);
    g();
}

void g() {
    std::lock_guard<std::mutex> lock(m); // deadlock
}
```

Почему deadlock?

Поток уже держит `m`, но пытается снова его захватить.

Обычный `std::mutex` не знает, что это тот же поток. Он просто видит, что мьютекс занят.

22. Как работает `std::recursive_mutex`

`std::recursive_mutex` позволяет одному и тому же потоку захватывать мьютекс несколько раз.

Внутри концептуально есть:

```
owner thread id
recursion counter
```

Схема:

```
thread #1 lock()   counter = 1
thread #1 lock()   counter = 2
thread #1 unlock() counter = 1
thread #1 unlock() counter = 0 → mutex freed
```

Пример:

```
#include <mutex>

class Tree {
public:
    void traverse() {
        std::lock_guard<std::recursive_mutex> lock(mutex_);

        // ...
        traverse_impl();
    }
}
```

```

    }

private:
    void traverse_impl() {
        std::lock_guard<std::recursive_mutex> lock(mutex_);

        // рекурсивная логика
    }

    std::recursive_mutex mutex_;
};

```

23. Почему `recursive_mutex` часто считается запахом дизайна

`std::recursive_mutex` бывает нужен, но часто он маскирует плохую архитектуру.

Например, если публичный метод берёт lock и вызывает приватный метод, который тоже берёт lock, лучше разделить:

```

class Counter {
public:
    int get() {
        std::lock_guard lock(mutex_);
        return get_unlocked();
    }

private:
    int get_unlocked() const {
        return value_;
    }

    int value_ = 0;
    std::mutex mutex_;
};

```

То есть вместо `recursive_mutex`:

публичные методы берут lock
 приватные `*_unlocked` методы предполагают, что lock уже взят

`recursive_mutex` может быть оправдан:

рекурсивные алгоритмы
 сложные callbacks

старый код
объектная модель, где трудно развести уровни lock-ов

Но на экзамене полезно сказать:

`recursive_mutex` решает самозахват мьютекса одним потоком, но его не стоит использовать без необходимости, потому что он может скрывать проблемы дизайна.

24. Deadlock

Deadlock — ситуация, когда потоки навсегда ждут друг друга.

Классический пример:

```
std::mutex m1;
std::mutex m2;

void thread1() {
    std::lock_guard<std::mutex> lock1(m1);

    // небольшая задержка для наглядности

    std::lock_guard<std::mutex> lock2(m2);
}

void thread2() {
    std::lock_guard<std::mutex> lock2(m2);

    // небольшая задержка для наглядности

    std::lock_guard<std::mutex> lock1(m1);
}
```

Сценарий:

```
thread #1 захватил m1
thread #2 захватил m2
thread #1 ждёт m2
thread #2 ждёт m1
```

Схема:

```
thread #1: holds m1 → waits m2
thread #2: holds m2 → waits m1
```

никто не может продолжить

25. Как предотвращать deadlock

Основные способы:

1. Всегда захватывать мьютексы в одном порядке

Например:

сначала m1, потом m2

Во всех потоках.

```
void thread1() {
    std::lock_guard lock1(m1);
    std::lock_guard lock2(m2);
}

void thread2() {
    std::lock_guard lock1(m1);
    std::lock_guard lock2(m2);
}
```

2. Использовать `std::lock`

`std::lock` умеет захватывать несколько Lockable-объектов без deadlock.

```
std::mutex m1;
std::mutex m2;

void f() {
    std::unique_lock<std::mutex> lock1(m1, std::defer_lock);
    std::unique_lock<std::mutex> lock2(m2, std::defer_lock);

    std::lock(lock1, lock2);

    // оба мьютекса захвачены
}
```

Почему нужен `std::defer_lock`?

Потому что мы создаём `unique_lock`, но не захватываем мьютексы сразу.

```
std::unique_lock<std::mutex> lock1(m1, std::defer_lock);
std::unique_lock<std::mutex> lock2(m2, std::defer_lock);
```

Потом `std::lock` захватывает их вместе:

```
std::lock(lock1, lock2);
```

После выхода из области видимости оба `unique_lock` освободят свои мьютексы.

26. `std::lock` не значит “атомарно на уровне железа”

Фраза “захватывает несколько мьютексов атомарно” иногда вводит в заблуждение.

`std::lock` не делает одну волшебную атомарную инструкцию для двух мьютексов.

Он использует алгоритм, который избегает deadlock:

```
пытается захватить несколько мьютексов
если не получилось захватить все — отпускает уже захваченные
пробует снова
```

Идея:

```
all or nothing
```

То есть поток не остаётся навсегда в состоянии:

```
держу m1 и жду m2
```

если это может привести к deadlock.

27. `std::scoped_lock`

Начиная с C++17, часто проще использовать `std::scoped_lock`.

```
std::mutex m1;
std::mutex m2;

void f() {
    std::scoped_lock lock(m1, m2);
```

```
    // оба мьютекса захвачены без deadlock  
}
```

`std::scoped_lock`:

RAII-объект
может захватывать несколько мьютексов
использует deadlock avoidance
освобождает все мьютексы в деструкторе

Для современного C++ это обычно самый удобный способ захватить несколько мьютексов.

28. `std::adopt_lock`

Иногда встречается такой паттерн:

```
std::lock(m1, m2);  
  
std::lock_guard<std::mutex> lock1(m1, std::adopt_lock);  
std::lock_guard<std::mutex> lock2(m2, std::adopt_lock);
```

Что значит `adopt_lock`?

“Мьютекс уже захвачен, `lock_guard` должен только взять ответственность за `unlock` в деструкторе.”

То есть `lock_guard` не вызывает `lock()` в конструкторе, а только “усыновляет” уже захваченный мьютекс.

Но в современном коде чаще проще:

```
std::scoped_lock lock(m1, m2);
```

29. Livelock и priority inversion

В программе явно в этом билете перечислены только мьютексы, но в конспекте рядом упоминаются и другие проблемы многопоточности.

Livelock

Потоки не заблокированы формально, но постоянно уступают друг другу и не делают полезной работы.

Пример из жизни:

два человека в коридоре одновременно отходят в одну сторону,
потом одновременно в другую,
и так несколько раз

В коде livelock может возникать, если потоки постоянно отпускают ресурсы и пробуют снова, но синхронно мешают друг другу.

Priority inversion

Высокоприоритетный поток ждёт мьютекс, который держит низкоприоритетный поток.

Если среднеприоритетные потоки вытесняют низкоприоритетный, высокоприоритетный тоже косвенно ждёт среднеприоритетные.

Схема:

Low priority thread держит mutex
High priority thread ждёт mutex
Medium priority threads мешают Low продолжить

Решения на уровне ОС могут включать priority inheritance.

Для базового ответа достаточно упомянуть как возможную проблему блокировок.

30. Что защищает mutex, а что не защищает

Очень важная мысль:

Мьютекс сам по себе не защищает данные. Данные защищает соглашение: все обращения к этим данным должны происходить под одним и тем же мьютексом.

Плохо:

```
std::mutex m;  
int counter = 0;  
  
void safe_increment() {  
    std::lock_guard lock(m);  
    ++counter;  
}  
  
int unsafe_get() {
```

```
    return counter; // чтение без lock
}
```

Даже если запись защищена, чтение без lock может быть data race.

Правильно:

```
int safe_get() {
    std::lock_guard lock(m);
    return counter;
}
```

Идея:

shared data + associated mutex

Нужно всегда понимать:

какой мьютекс защищает какие данные

31. Не держать lock во время I/O

Плохой пример:

```
std::mutex cache_mutex;
std::unordered_map<long long, bool> cache;

void handle_client(int client_fd, long long number) {
    std::lock_guard lock(cache_mutex);

    bool result = get_or_compute(number);

    send_result(client_fd, result); // плохо: сетевой I/O под mutex
}
```

Почему плохо?

send_result может заблокироваться
другие потоки не смогут работать с cache
один медленный клиент тормозит всех

Лучше:


```

bool result;

{
    std::lock_guard lock(cache_mutex);
    result = get_cached_value(number);
}

send_result(client_fd, result);

```

Или если нужно вычислять:

```

bool result = get_result(number);

send_result(client_fd, result);

```

где `get_result` сам минимизирует критическую секцию.

32. Потокобезопасность интерфейса

Иногда кажется, что если каждый метод класса защищён мьютексом, всё хорошо.

Пример:

```

class SafeVector {
public:
    bool empty() const {
        std::lock_guard lock(mutex_);
        return data_.empty();
    }

    int back() const {
        std::lock_guard lock(mutex_);
        return data_.back();
    }

    void pop_back() {
        std::lock_guard lock(mutex_);
        data_.pop_back();
    }

private:
    mutable std::mutex mutex_;
    std::vector<int> data_;
};

```

На первый взгляд каждый метод безопасен.

Но такой код всё равно может быть логически опасен:

```
if (!v.empty()) {  
    int x = v.back();  
    v.pop_back();  
}
```

Между `empty()` и `back()` другой поток может изменить вектор.

Вывод:

потокобезопасные отдельные методы не всегда дают потокобезопасную операцию

Иногда нужно делать одну составную операцию под одним lock:

```
std::optional<int> try_pop() {  
    std::lock_guard lock(mutex_);  
  
    if (data_.empty()) {  
        return std::nullopt;  
    }  
  
    int x = data_.back();  
    data_.pop_back();  
  
    return x;  
}
```

Это важное профессиональное дополнение.

33. Мини-пример: потокобезопасный счётчик

```
class Counter {  
public:  
    void increment() {  
        std::lock_guard lock(mutex_);  
        ++value_;  
    }  
  
    int get() const {  
        std::lock_guard lock(mutex_);  
        return value_;  
    }  
  
private:
```

```
mutable std::mutex mutex_;
int value_ = 0;
};
```

Почему `mutex_` помечен как `mutable`?

Потому что метод `get()` логически не меняет значение счётчика, но должен захватить мьютекс.

```
int get() const
```

внутри вызывает:

```
mutex_.lock();
```

А это формально меняет состояние объекта `mutex_`.

34. Мини-пример: потокобезопасный кэш

```
class PrimeCache {
public:
    bool get(long long number) {
        {
            std::shared_lock lock(mutex_);

            auto it = cache_.find(number);

            if (it != cache_.end()) {
                return it->second;
            }
        }

        bool result = is_prime(number);

        {
            std::unique_lock lock(mutex_);

            auto [it, inserted] = cache_.emplace(number, result);

            return it->second;
        }
    }

private:
    static bool is_prime(long long n) {
        if (n <= 1) {
```

```

        return false;
    }

    if (n == 2) {
        return true;
    }

    if (n % 2 == 0) {
        return false;
    }

    for (long long d = 3; d * d <= n; d += 2) {
        if (n % d == 0) {
            return false;
        }
    }

    return true;
}

std::unordered_map<long long, bool> cache_;
mutable std::shared_mutex mutex_;
};

```

35. Что сказать на экзамене: короткая версия

Можно отвечать так:

В многопоточном коде проблема возникает, когда несколько потоков обращаются к общим изменяемым данным. Если хотя бы один поток пишет, а синхронизации нет, возникает data race, что в C++ является undefined behavior. Например, если несколько потоков одновременно читают и изменяют общий `unordered_map`, его внутренняя структура может быть повреждена.

Базовый способ защиты — `std::mutex`. Он обеспечивает взаимное исключение: только один поток может находиться в критической секции. Обычно мьютекс используют не через ручные `lock/unlock`, а через RAII-объекты. `std::lock_guard` захватывает мьютекс в конструкторе и освобождает в деструкторе.

Если нужна большая гибкость, используют `std::unique_lock`: его можно явно `unlock / lock`, можно создать с `std::defer_lock`, можно перемещать. Он нужен, например, для `condition_variable` и для захвата нескольких мьютексов через `std::lock`.

Если структура имеет много чтений и мало записей, можно использовать `std::shared_mutex`. Для чтения берётся `std::shared_lock`, и несколько

читателей могут работать параллельно. Для записи берётся `std::unique_lock`, и писатель получает эксклюзивный доступ.

`std::recursive_mutex` позволяет одному и тому же потоку захватывать мьютекс несколько раз. Внутри у него есть счётчик рекурсии, но использовать его стоит осторожно, потому что часто он скрывает проблемы дизайна.

Deadlock возникает, если потоки захватывают несколько мьютексов в разном порядке. Например, один поток держит `m1` и ждёт `m2`, а другой держит `m2` и ждёт `m1`. Чтобы избежать этого, можно всегда захватывать мьютексы в одном порядке или использовать `std::lock` / `std::scoped_lock`, которые захватывают несколько мьютексов без deadlock.

36. Мини-код для ответа на доске

`std::mutex`

```
std::unordered_map<long long, bool> cache;
std::mutex cache_mutex;

bool get_result(long long number) {
    std::lock_guard<std::mutex> lock(cache_mutex);

    auto it = cache.find(number);

    if (it != cache.end()) {
        return it->second;
    }

    bool result = is_prime(number);
    cache[number] = result;

    return result;
}
```

`std::shared_mutex`

```
std::unordered_map<long long, bool> cache;
std::shared_mutex cache_mutex;

bool get_result(long long number) {
    {
        std::shared_lock lock(cache_mutex);

        auto it = cache.find(number);
```

```

        if (it != cache.end()) {
            return it->second;
        }
    }

    bool result = is_prime(number);

    {
        std::unique_lock lock(cache_mutex);

        auto [it, inserted] = cache.emplace(number, result);

        return it->second;
    }
}

```

std::lock

```

std::mutex m1;
std::mutex m2;

void f() {
    std::unique_lock<std::mutex> lock1(m1, std::defer_lock);
    std::unique_lock<std::mutex> lock2(m2, std::defer_lock);

    std::lock(lock1, lock2);

    // оба мьютекса захвачены
}

```

Современный вариант:

```

void f() {
    std::scoped_lock lock(m1, m2);

    // оба мьютекса захвачены
}

```

37. Мини-проверка

1. В каком случае возникает проблема доступа к общим данным?
2. Что такое data race?
3. Почему data race в C++ опасен?
4. Чем data race отличается от race condition?
5. Почему `++counter` не является потокобезопасным?

6. Что такое критическая секция?
7. Что гарантирует `std::mutex` ?
8. Почему лучше не писать вручную `mutex.lock()` и `mutex.unlock()` ?
9. Что делает `std::lock_guard` ?
10. Что такое RAII?
11. Почему мьютекс должен защищать все обращения к данным, а не только записи?
12. Почему нельзя держать мьютекс во время долгого I/O?
13. Чем `std::unique_lock` отличается от `std::lock_guard` ?
14. В каких случаях нужен `std::unique_lock` ?
15. Что такое `std::shared_mutex` ?
16. Чем отличается `std::shared_lock` от `std::unique_lock` ?
17. Когда `shared_mutex` может быть полезнее обычного `mutex` ?
18. В чём минусы `shared_mutex` ?
19. Что такое `std::recursive_mutex` ?
20. Почему `recursive_mutex` может быть признаком плохого дизайна?
21. Что такое deadlock?
22. Приведи пример deadlock с двумя мьютексами.
23. Как можно избежать deadlock?
24. Что делает `std::lock` ?
25. Зачем нужен `std::defer_lock` ?
26. Чем `std::scoped_lock` удобнее ручного `std::lock` ?
27. Что значит `std::adopt_lock` ?
28. Почему потокобезопасные отдельные методы класса не всегда дают потокобезопасную составную операцию?
29. Что такое livelock?
30. Что такое priority inversion?

38. Самый короткий ответ, если времени совсем мало

Проблема доступа к общим данным возникает, когда несколько потоков обращаются к одной изменяемой структуре, например кэш или счётчику. Если хотя бы один поток пишет, а синхронизации нет, возникает data race, что в C++ является undefined behavior.

`std::mutex` обеспечивает взаимное исключение: только один поток может находиться в критической секции. Обычно его используют через RAII: `std::lock_guard` или `std::unique_lock`. `lock_guard` простой и захватывает мьютекс на время области видимости. `unique_lock` гибче: его можно явно unlock/lock, создавать с `defer_lock`, перемещать и использовать с condition variable.

`std::shared_mutex` позволяет нескольким читателям работать параллельно через `std::shared_lock`, но писатель берёт эксклюзивный `std::unique_lock`. Это полезно, когда чтений много, а записей мало.

`std::recursive_mutex` позволяет одному потоку захватить один и тот же мьютекс несколько раз, но часто это признак неудачного дизайна.

При захвате нескольких мьютексов возможен deadlock. Чтобы его избежать, нужно захватывать мьютексы в одном порядке или использовать `std::lock` / `std::scoped_lock`, которые захватывают несколько мьютексов без взаимной блокировки.